CHAPTER 2 · SEARCH SEMANTICALLY

Keyword / Lexical Search

Match the words — fast. The workhorse behind decades of search.

Dr. Mahdi Habibi

WHERE WE ARE

Connect a query to the right documents — in milliseconds

- A user gives a query; we must return the documents that match.
- It has to be fast — potentially over millions of documents.
- Keyword search scores documents by their shared words.
- Two ideas make it work: an index (speed) + a scoring function (quality).

INTUITION

Start simple: count the shared words

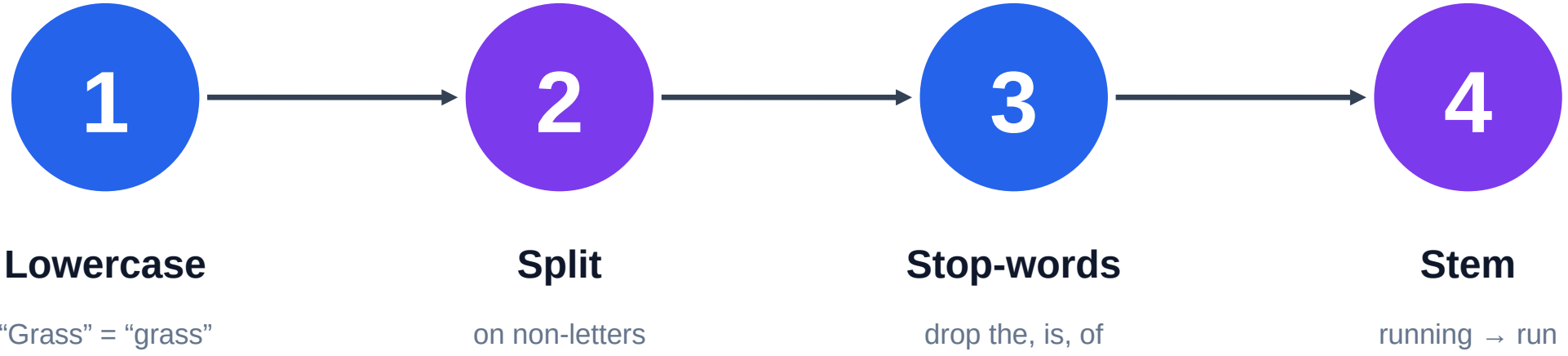
Query: *what color is the grass*

D0	the grass is green	3
D1	the sky is blue	2
D2	the capital of canada is ottawa	2
D3	a whale is a mammal	1

D0 wins — it repeats the query's words. But common words like “the” and “is” dominate the count. We need to weight them.

STEP 1

Tokenization: turn text into comparable tokens

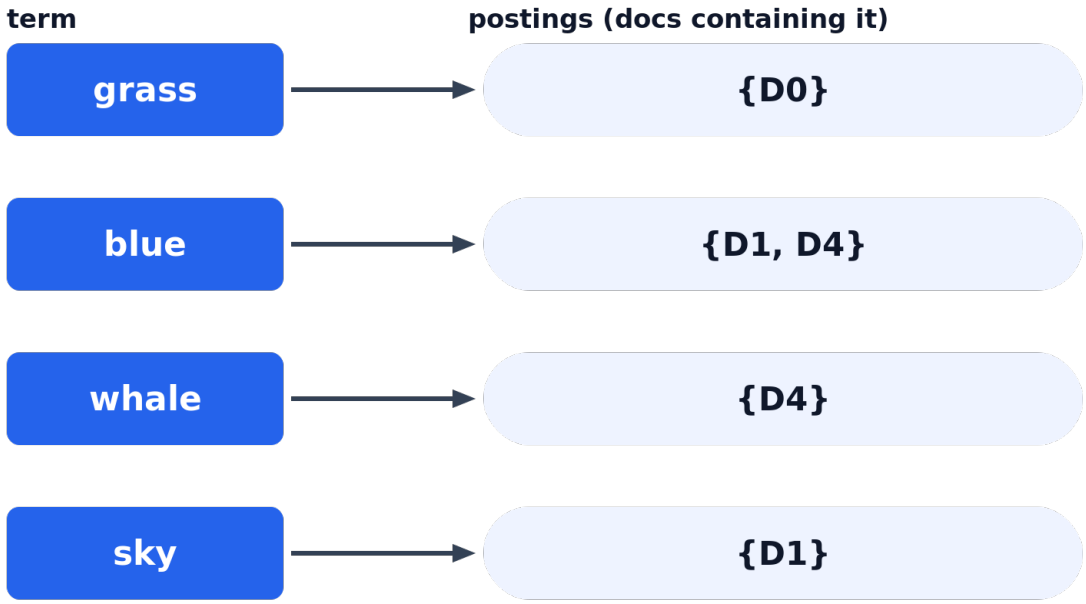


Tokenize the query the SAME way as the documents — or matches silently vanish.

STEP 2

The inverted index: how search is instant

- Map every term → the documents that contain it.
- Built once, ahead of time (at indexing).
- A query becomes a single lookup, not a full scan.
- This is why results come back in milliseconds.



STEP 3

TF-IDF: weight words by how informative they are

TF

Term Frequency

Frequent in a document → probably matters to it.

IDF

Inverse Doc Frequency

Rare across the corpus → more distinctive & useful.

TF-IDF = TF × IDF — frequent here AND rare overall = a high score.

$$\mathrm{IDF}(t) = \ln \frac{N}{\mathrm{df}(t)}$$

WORKED EXAMPLE

Why “is” scores nothing and “grass” wins

term	df (docs with it)	IDF = ln(N / df)
is	5 of 5	0.00
the	3 of 5	0.51
grass	1 of 5	1.61

score(D0) ≈ 2.12

driven almost entirely by “grass”.
“is” is in every doc → IDF 0 → adds nothing.

N = 5 documents. IDF = ln(N / df): common terms → ~0, rare terms → large.

STEP 4

BM25: the workhorse ranking function

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

k1 — term saturation

b — length normalization

Diminishing returns for repeated terms, and fairness for long documents. The default first-stage ranker for decades — fast, robust, no training.

PITFALLS & GOTCHAS

Where keyword search goes wrong

- **Mismatched tokenization** — Query and docs must be normalized the same way, or matches disappear.
- **Over-aggressive stop-words** — Dropping “not” or “no” can flip meaning.
- **Raw counts, no length norm** — Long documents dominate. Use BM25, not plain TF.
- **Expecting synonyms** — Keyword search can't match meaning — that needs embeddings (Ch 3–6).

KEY TERMS

The vocabulary of keyword search

lexical search | tokenization | stop words | stemming | inverted index | postings

term frequency (TF) | document frequency (df) | IDF | TF-IDF | BM25 | term saturation

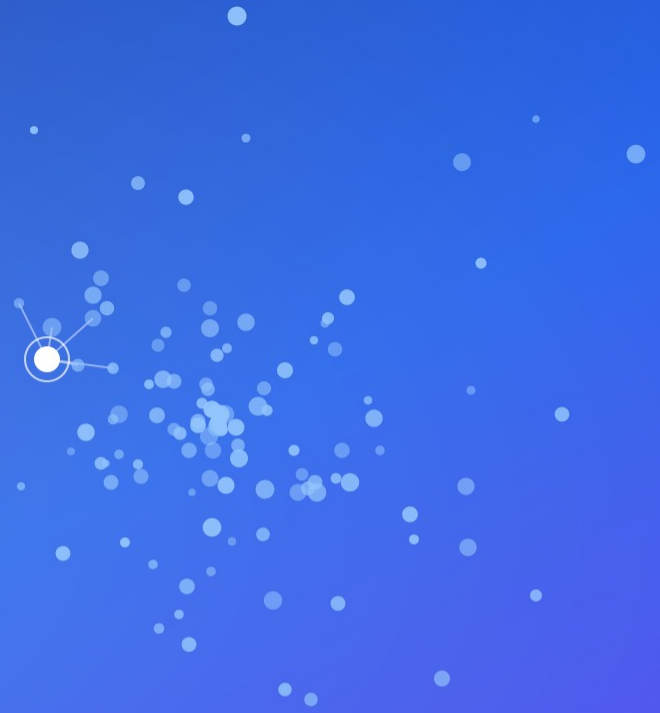
length normalization

Full definitions in the course GLOSSARY.

CHECK YOUR UNDERSTANDING

Test yourself, then move on

- Why is it called an inverted index, and what problem does it solve?
- In the worked example, why does “is” contribute nothing?
- What two problems does BM25 fix, and which parameter controls each?



Next → Chapter 3: From Text to Vectors